



软件体系结构

第2章 软件的生命周期

赵庆玲

南京理工大学计算机科学与工程学院

ada_zhao@njust.edu.cn



1、引言

(1)什么是软件工程

定义1： 软件工程是指导计算机软件开发和维护的工程学科。采用工程的概念、原理、技术和方法来开发与维护软件，把经过时间考验而证明正确的管理技术和当前能够得到的最好的技术方法结合起来，这就是软件工程。

定义2： 软件工程是研究和应用如何以系统性的、规范化的、可量化的方法去开发、操纵和维护软件、即把工程应用到软件上。

(2)什么是软件生命期

定义3： 软件生命期是指软件产品从考虑其概念开始，到该软件产品不再能使用为止的整个时期。一般包括概念阶段、需求阶段、设计阶段、实现阶段、测试阶段、安装阶段以及交付使用阶段、运行阶段和维护阶段。有时还有退役阶段。这些阶段可以有重复，执行时也可以有迭代。

(3)什么是软件开发生命期

定义4：软件开发生命期是指软件产品从考虑其概念开始到该软件产品交付使用使用为止的整个时期。一般包括概念阶段、需求阶段、设计阶段、实现阶段、测试阶段、安装阶段，以及交付阶段。这些阶段可以有重叠，执行时也可以有迭代。

(4)什么是软件开发过程

定义5：把用户的要求转变成软件产品的过程叫做软件开发过程。此过程包括对用户的要求进行分析，解释成软件需求，把需求变换成设计，把设计用代码来实现，测试该代码，有时还要进行代码安装和把软件交付运行使用。注意：这些活动可以重叠，执行时也可以有迭代。

定义6：进行一组有组织的活动以把用户的要求转化成软件产品。



2、软件开发模型的发展

在整个软件开发的发展过程中，为了要从宏观上管理软件的开发和维护，就必须对软件的发展过程有总体的认识和描述，即要对软件过程建模。几十年来，软件开发生命周期模型的发展有了很大变化，提出了一系列的模型以适应软件开发发展的需要。在这里我们提出以下一些主要的软件开发模型：

1. 编码—修正模型(code and fix model)



2、软件开发模型的发展

弊端：

首先，代码缺少统一规划，低估了设计的重要性，使得代码结构随着修改次数的增加变得越来越坏。以至错误越来越难改，甚至无法改。

第二，即使有的软件设计得很好，但往往其结果并非用户所需要的。造成软件开发的风险非常大。这主要是因为没有重视需求而造成的。

第三，由于对测试、维护修改方面考虑不周，使得代码的维护修改非常困难。

所以，当开发的软件规模不断扩大时，这种开发模型就会引起严重的后果，必须加以改进。



2、软件开发模型的发展

2. 瀑布模型(waterfall model)

由于吸取软件开发早期的教训，人们开始将软件开发视为工程来管理。类似其他工程的管理，软件开发也具有一定的工序。于是，“软件生命周期”这一概念真正被提了出来，并将软件生命周期划分成了：制定计划、需求分析和定义、软件设计、程序编写、软件测试、运行和维护这六个步骤。

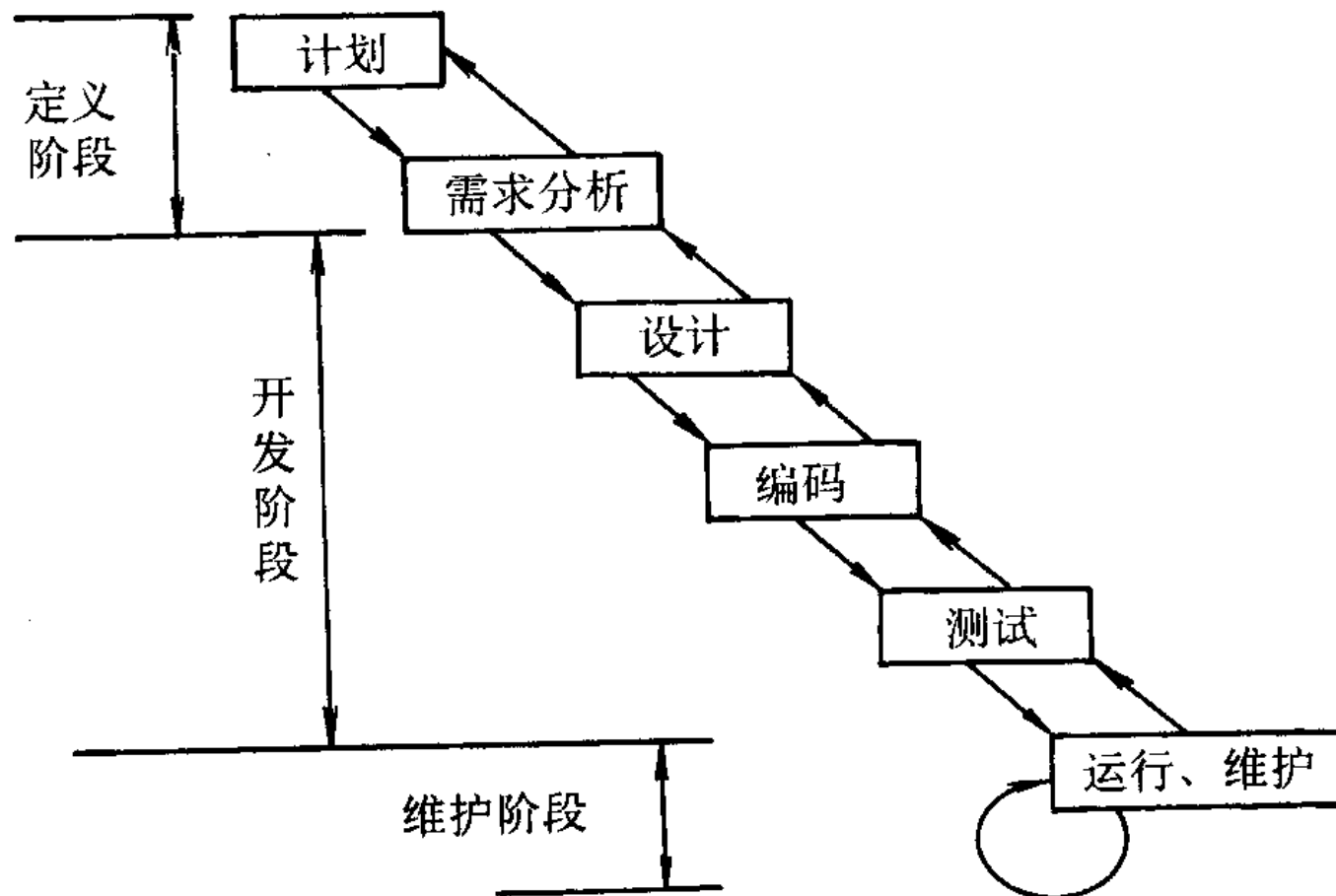
在这一基础上，Winston W. Royce在1970年提出了著名的“瀑布模型”。瀑布模型规定了包括上述六个工程活动，并且规定了它们自上而下、相互衔接的固定次序，如同瀑布流水，逐级下落，并试图解决编码—修正模型所带来的问题。然而软件开发的实践表明，上述各项活动之间并非完全是自上而下，呈线性因式。实际情况中，每项开发活动大部分具有以下特点：

2、软件开发模型的发展



- (1)从上一项开发活动接受该项活动的工作对象，作为输入。
- (2)利用这一输入，实施该项活动应完成的工作内容。
- (3)给出该项活动的工作成果，作为输出传给下一项开发活动。
- (4)对该项活动的实施工作成果进行评审。若其工作成果得到确认，则继续进行下一项开发活动，如下图中的向下箭头所表示；否则返回前一项，甚至更前项的活动。尽量减少多个阶段间的反复。以相对来说较小的费用来开发软件。

2、软件开发模型的发展





2、软件开发模型的发展

但是瀑布模型同样存在一些问题：

首先，阶段和阶段划分完全固定，阶段间产生大量的文档，极大地增加了工作量。

第二，由于开发模型呈线性，所以当开发成果尚未经过测试时，用户无法看到软件的效果。这样，软件与用户见面的时间间隔较长，也增加了一定的风险。

第三，前面未发现的错误传到后面的开发活动中时，可能会扩散，进而可能会造成更不理想的后果。

为此，常常在需求阶段或设计阶段平行地进行几次快速原型，来消除风险和不确定性。该模型长期以来已成为美国政府或军方的软件项目的主要标准。但是瀑布模型的一些根本性缺陷推动着人们继续探索新的模型。相当一部分人认为应考虑采用形式化方法的转换模型以减少人为的错误。



2、软件开发模型的发展

3. 转换模型

其主要思想是用形式化的方法自动生成程序。主要步骤为：

(1)采用形式化的规格说明书。(2)通过自动系统自动地变换成代码。(3)必要时做一些优化，改进性能。(4)交付用户使用。(5)根据使用的经验来调整形式化的规格说明书。返回(1)重复整个过程。

转换模型的优点是解决了代码结构经多次修改而变坏的问题；减少了许多中间步骤，如设计、编码、测试等等。但是，转换模型仍有较大局限，表现在：

(1)自动转换在实际上仅适用于规模很小产品；或某些非常特定领域的产品。(2)用户容易无计划地修改，是否能保证向正确、优化及改进方向发展呢？

(3)需要非常庞大的支持体系。特别考虑到当前技术飞速发展的特点，这种进行自动转换的系统中应该包含和维护的知识数量呈爆炸性(商品化的软件，硬件平台知识)。

4. 平行瀑布模型

现在考虑对瀑布模型的进一步改进。对瀑布模型的各个阶段之间转换时，不一定要完全按顺序进行。而是可以适当并行开展各阶段的开发工作。在上一个阶段尚未完全结束前，就可开设后一阶段的开发工作。例如，在需求分析完成60%时，就可开始进行这60%的已完成分析部分的设计工作。同时并行进行余留的40%的需求分析。根据不同情况可有不同的并行度。例如：

(1)用户想法不稳定，比方说，每天变换想法，要求不太清楚的话，则增加并行度。

(2)短期显示成果的压力大，则可增加并行度。例如，在某种场合，在某些人的眼光中，按期完成70%的测试后的程序，比做完100%的设计，但无一行代码的印象好得多。

(3)如果可靠性要求高；要求各方面控制和配合很严格；资源及预算严密；技术错误的后果严重时，则需减少并行度。

一般，对小系统关系不大，但对于大型系统的开发，则需根据实际情况认真分析考虑，难以用一个固定衡量标准。

5. 演进式开发模型

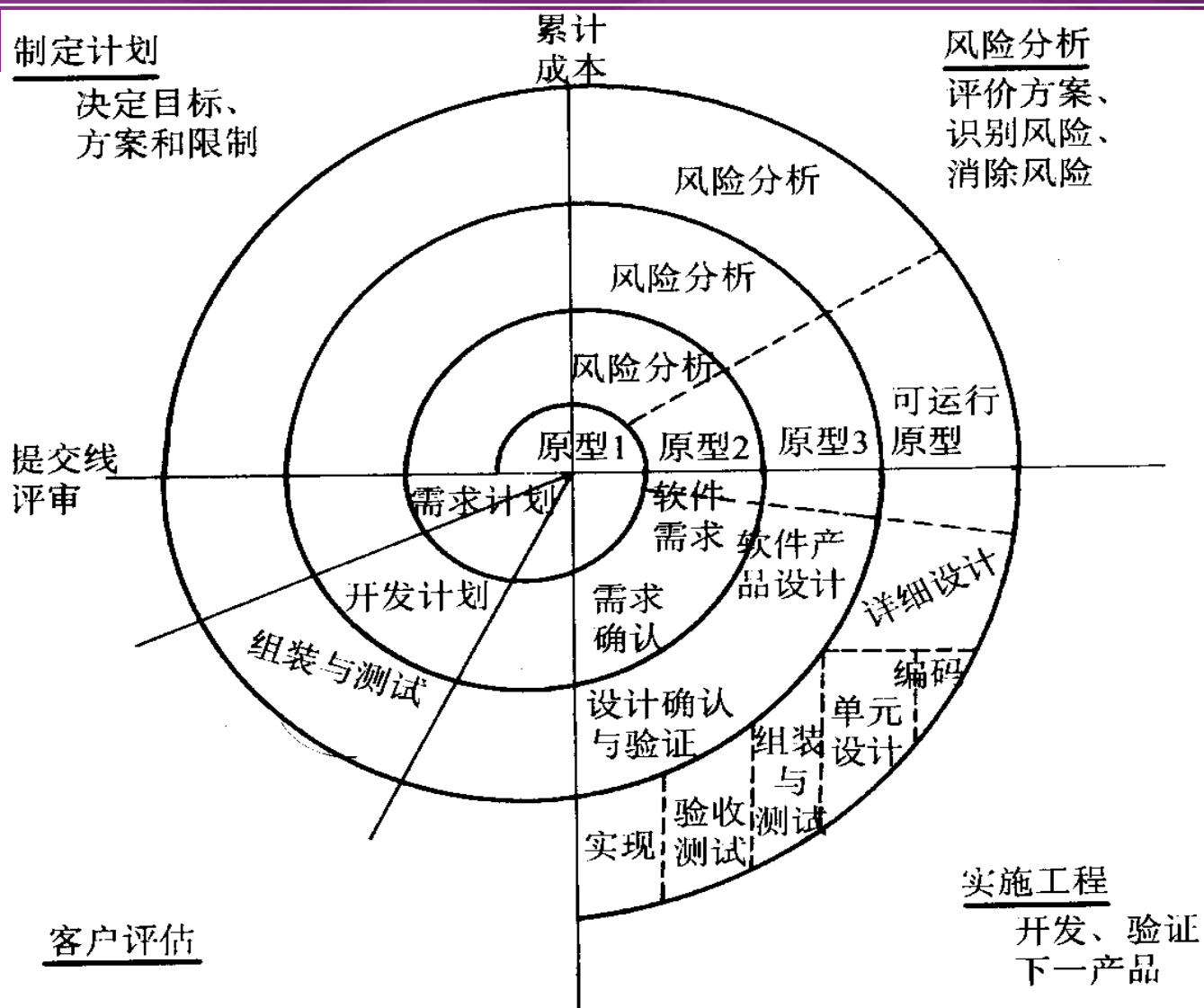
由于在项目开发的初期，人们对软件需求的认识常常不够清晰，因而使得在进行项目开发时，难于做到一次开发成功。出现返工，再一次开发常常在所难免。有人说，目标只在于探索可行性，弄清软件需求；第二次则在第一次开发的基础上获得较为满意的软件产品。通常把第一次得到的试验性产品称为“原型”。如果系统十分复杂，就可能要开发多次，每次只开发系统的一部分功能。每次在前一次的基础上，进行改进并扩大性能、功能，直到达到要求。显然，这种依靠演进方式进行开发的演化模型在克服瀑布模型的缺点，减少由于软件需求不明确而给工作带来风险方面，确有显著的效果。这种开发模型的问题是不要把演进式开发模型实际执行成原始的编码—修正模型。

6. 螺旋模型

70年代和80年代，硬件技术不断提高，软件系统也不断庞大。而当我们面对一个复杂的大型软件系统时，用“瀑布模型”还是“演化模型”都难于有效地完成项目。于是，

Barry Boehm在1988年正式发表了软件系统开发的“螺旋模型”。螺旋模型将瀑布模型和演化模型等结合起来，并且强调了其他模型均忽略了的风险分析。这种模型特别适用于庞大而复杂的系统、高风险的系统。对于这些系统，风险是软件开发不可忽视的、潜在的不利因素。它可能在不同程度上损害软件开发过程或软件产品的质量。减小软件风险的目标是在造成危害之前，及时对风险进行识别、分析，决定采取何种对策，进而消除或减少风险的损害。

螺旋模型如图所示。





螺旋模型更适合于大型软件的开发，应该说它对于具有高度风险的大型复杂软件系统的开发是最为实际的方法。螺旋模型实际上是吸收和综合了过去各种软件开发模型。每种模型都可用特定情况的螺旋模型来表示。但它强调风险分析，吸收了软件工程“演化”的概念，使得开发人员和客户对每个演化层出现的风险有所了解，继而做出应有的反应。但是，还不能说螺旋模型绝对比其他模型优越。首先，要求许多客户接受和相信演化方法并不容易。其次，需要具有相当丰富的风险评估经验和专门知识，才能使用这个模型。对于风险较大的项目来说，在这类项目开发中，如果未能够及时发现风险，势必会造成重大损失。采用螺旋模型显然是很合适的。

各种模型的比较



- 每个软件开发组织应该选择适合于该组织的软件开发模型，并且应该随着当前正在开发的特定产品特性而变化，以减小所选模型的缺点，充分利用其优点。

模型	优点	缺点
瀑布模型	文档驱动	系统可能不满足客户的需求
快速原型模型	关注满足客户需求	可能导致系统设计差、效率低，难于维护
增量模型	开发早期反馈及时，易于维护	需要开放式体系结构，可能会设计差、效率低
螺旋模型	风险驱动	风险分析人员需要有经验且经过充分训练